

FACULDADE IMPACTA TECNOLOGIA

BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

APOSTILA DE PARADIGMAS DE PROGRAMAÇÃO

Linguagens Imperativas

Material didático para graduação — foco em C

Docente: Prof. Alex Torquato Souza Carneiro

São Paulo — 2026/2

Apresentação

Esta apostila introduz o paradigma imperativo a partir da ideia de que um programa é uma sequência de comandos que transforma um estado computacional. O percurso combina fundamentos de linguagens de programação com práticas em C, linguagem especialmente adequada para observar variáveis, memória, endereços e estruturas de dados.

O material foi alinhado ao plano de ensino da disciplina Paradigmas de Programação (80 horas, com equilíbrio entre teoria e prática) e aos tópicos discutidos em Programming Languages — Principles and Paradigms, de Maurizio Gabbrielli e Simone Martini, especialmente controle, abstração, tipos compostos e aspectos históricos das linguagens.

Objetivos de aprendizagem

- Explicar o que caracteriza uma linguagem imperativa e distingui-la de linguagens declarativas.
- Descrever estado, memória, ambientes, variáveis, atribuição e efeitos colaterais.
- Usar tipos básicos e compostos de C com segurança e clareza.
- Implementar funções, passagem de parâmetros, ponteiros, structs e listas encadeadas.
- Analisar decisões de projeto: legibilidade, eficiência, confiabilidade, segurança e portabilidade.
- Relacionar o paradigma imperativo a abordagens orientada a objetos, funcional e lógica.

Como estudar

Leia a síntese conceitual, execute os exemplos, modifique uma hipótese por vez e registre o comportamento observado.

Sumário

1. O paradigma imperativo	3
2. Anatomia de um programa em C	5
3. Controle de fluxo e programação estruturada	7
4. Arrays e strings	9
5. Funções e abstração de controle	11
6. Ponteiros, endereços e memória dinâmica	13
7. Structs e listas encadeadas	15
8. Comparação com outros paradigmas	17
9. Projeto integrador	18
10. Exercícios de revisão	19
Glossário	22
Referências	23

1. O paradigma imperativo

Em uma linguagem imperativa, a computação é descrita como uma sucessão de ações. A operação central é a atribuição: calcular uma expressão e armazenar o resultado em uma localização associada a uma variável. Assim, o significado de um programa não depende apenas dos valores produzidos, mas também da sequência em que o estado é alterado.

1.1 Estado, ambiente e memória

Uma forma útil de separar conceitos é distinguir ambiente e memória. O ambiente associa nomes a entidades; a memória associa localizações a valores armazenáveis. Uma variável pode ser vista como um nome que designa uma localização cujo conteúdo pode mudar ao longo da execução.

Conceito	Pergunta orientadora	Exemplo em C
Ambiente	O que este nome designa?	<code>int total;</code>
Memória	Onde o valor está armazenado?	<code>&total</code>
Estado	Quais valores estão nas localizações?	<code>total == 42</code>
Atribuição	Como o estado muda?	<code>total = total + 1;</code>
Efeito colateral	O que foi alterado além do valor retornado?	função que modifica um argumento

1.2 Características e vantagens

- Sequência explícita de comandos e controle estruturado.
- Variáveis modificáveis e atribuição como operação fundamental.
- Proximidade do modelo de execução da máquina, especialmente em C.
- Boa adequação a sistemas, embarcados, compiladores, jogos e estruturas de dados.
- Possibilidade de controle fino de representação e uso de memória.

1.3 Critérios de avaliação de linguagens

Ao comparar linguagens, considere legibilidade, facilidade de escrita, confiabilidade, custo, eficiência, portabilidade, segurança e suporte à manutenção. C oferece eficiência e controle, mas transfere ao programador responsabilidades que linguagens com gerenciamento automático de memória ou verificações mais fortes podem assumir.

Atividade de reflexão

1. Em que sentido a atribuição é uma mudança de estado?
2. Por que duas expressões com o mesmo valor final podem ter comportamentos diferentes quando produzem efeitos colaterais?
3. Cite uma situação em que o controle explícito da memória é uma vantagem e outra em que é um risco.

2. Anatomia de um programa em C

C é uma linguagem compilada, de tipagem estática e com recursos imperativos centrais. Um programa é organizado em funções, declarações, expressões e comandos. O ponto de entrada convencional é a função `main`.

```
#include <stdio.h>

int main(void) {
    int x = 2;
    x = x + 3;
    printf("x = %d\n", x);
    return 0;
}
```

2.1 Tipos básicos e representação

Tipos definem quais valores podem ser representados e quais operações são permitidas. Em C, `int` representa inteiros, `char` caracteres/pequenos inteiros, `float` e `double` números de ponto flutuante, e `_Bool` valores lógicos. O tamanho exato de alguns tipos depende da implementação; quando portabilidade importa, use `<stdint.h>`, como `int32_t`.

Tipo	Uso típico	Atenção
<code>int</code>	contagens e índices	overflow de inteiros assinados é comportamento indefinido
<code>double</code>	medidas e cálculos reais	igualdade exata pode ser inadequada
<code>char</code>	caracteres e bytes	<code>char</code> pode ser signed ou unsigned
<code>_Bool</code>	condições	inclua <code><stdbool.h></code> para <code>bool</code> , <code>true</code> e <code>false</code>
<code>size_t</code>	tamanhos e índices	tipo sem sinal, adequado a <code>sizeof</code>

2.2 Expressões, precedência e conversões

A expressão $2 + 3 * 4$ vale 14 porque a multiplicação tem precedência maior. Parênteses melhoram a intenção. Conversões implícitas podem alterar resultados; em cálculos mistos, torne a conversão explícita e evite comparar diretamente valores de ponto flutuante sem tolerância.

2.3 Entrada e saída

```
#include <stdio.h>

int main(void) {
    int idade;
    printf("Idade: ");
    if (scanf("%d", &idade) == 1) {
        printf("Você informou %d.\n", idade);
    }
    return 0;
}
```

3. Controle de fluxo e programação estruturada

Comandos sequenciais, condicionais e iterativos organizam o fluxo de execução. Blocos delimitados por chaves agrupam comandos. A programação estruturada privilegia estruturas com entrada e saída compreensíveis, reduzindo saltos arbitrários e favorecendo testes e manutenção.

3.1 Seleção

```
if (media >= 6.0) {
    printf("Aprovado\n");
} else {
    printf("Reprovado\n");
}
```

3.2 Repetição

```
int soma = 0;
for (int i = 1; i <= 10; i++) {
    soma += i;
}
printf("%d\n", soma);
```

Use `while` quando a quantidade de repetições depende de uma condição de entrada ou de parada; do ... `while` quando o corpo deve executar ao menos uma vez. `break` e `continue` são úteis em situações locais, mas seu uso excessivo pode ocultar o fluxo.

3.3 Laboratório 1 — média e classificação

Implemente um programa que leia três notas, calcule a média e mostre “Aprovado”, “Exame” ou “Reprovado” conforme faixas definidas pelo professor. Valide entradas inválidas, documente as decisões e teste valores de fronteira.

4. Arrays e strings

Um array é uma sequência contígua de elementos do mesmo tipo. Seus índices começam em zero e devem permanecer no intervalo válido. C não carrega automaticamente o tamanho do array, por isso o programa precisa manter essa informação.

```
int notas[5] = {7, 8, 6, 9, 10};
int soma = 0;
for (size_t i = 0; i < 5; i++) {
    soma += notas[i];
}
```

4.1 Strings em C

Uma string é um array de char terminado pelo caractere nulo \0. Funções como strlen, strcmp e snprintf pertencem a <string.h> e <stdio.h>. Prefira funções que recebem o tamanho do destino e sempre reserve espaço para o terminador.

```
char nome[32];
if (fgets(nome, sizeof nome, stdin) != NULL) {
    printf("Olá, %s", nome);
}
```

4.2 Riscos frequentes

- Acessar índice negativo ou maior que o último elemento.
- Esquecer o terminador nulo em strings.
- Usar scanf("%s", ...) sem limite de largura.
- Confundir sizeof(array) com sizeof(pointer).
- Assumir que arrays são copiados automaticamente ao passá-los para funções.

5. Funções e abstração de controle

Funções encapsulam uma sequência de comandos e permitem decompor problemas. Em C, parâmetros ordinários são passados por valor: a função recebe uma cópia. Para alterar um objeto do chamador, passe seu endereço por meio de um ponteiro.

```
int maximo(const int v[], size_t n) {
    int maior = v[0];
    for (size_t i = 1; i < n; i++) {
        if (v[i] > maior) maior = v[i];
    }
    return maior;
}
```

5.1 Contratos e recursão

Uma função bem definida explicita suas condições e efeitos colaterais. A recursão resolve um problema por meio de instâncias menores e exige caso-base. Quando a chamada recursiva não é a última operação, ainda pode haver trabalho pendente após o retorno.

```
unsigned long fatorial(unsigned n) {
    if (n <= 1) return 1;
    return n * fatorial(n - 1);
}
```

5.2 Laboratório 2 — estatísticas de um vetor

Crie funções para calcular mínimo, máximo, soma e média de um vetor. A função principal deve apenas coordenar a leitura, as chamadas e a apresentação. Inclua testes para vetor unitário, valores repetidos e entradas negativas.

6. Ponteiros, endereços e memória dinâmica

Um ponteiro armazena o endereço de um objeto ou função. O operador & obtém um endereço e * acessa o objeto apontado. A aritmética de ponteiros é definida em relação aos elementos de um array, e não como aritmética livre de bytes.

```
void trocar(int *a, int *b) {
    int temporario = *a;
    *a = *b;
    *b = temporario;
}
```

6.1 Vida útil e alocação

Objetos locais normalmente têm duração automática; objetos criados com malloc/calloc/realloc vivem até free. Todo recurso adquirido deve ter um caminho claro de liberação. Nunca use um ponteiro depois de liberar o objeto, nem retorne o endereço de uma variável local.

```
int *dados = malloc(10 * sizeof *dados);
if (dados == NULL) {
    return 1;
}
/* usar dados */
free(dados);
dados = NULL;
```

6.2 Checklist de segurança

- Inicialize ponteiros; use NULL quando não apontarem para objeto válido.
- Verifique o retorno de malloc, calloc, realloc e funções de entrada.
- Mantenha o tamanho junto com o bloco de memória.
- Evite double free, use-after-free, vazamento e acesso fora dos limites.
- Use const para indicar dados que a função não deve modificar.

7. Structs e listas encadeadas

struct reúne campos possivelmente de tipos diferentes. Em C, os campos são acessados com ponto quando se tem um objeto e com -> quando se tem um ponteiro para o objeto. Structs podem representar entidades e nós de estruturas recursivas.

```
typedef struct {
    int matricula;
    char nome[64];
    double media;
} Aluno;

Aluno a = {101, "Ana", 8.5};
printf("%s\n", a.nome);
```

7.1 Lista simplesmente encadeada

```
typedef struct No {
    int valor;
    struct No *proximo;
} No;

void inserir_inicio(No **lista, int valor) {
    No *novo = malloc(sizeof *novo);
    if (!novo) return;
    novo->valor = valor;
    novo->proximo = *lista;
    *lista = novo;
}
```

O ponteiro No ** permite que a função altere o ponteiro inicial da lista. Ao percorrer a estrutura, avance com atual = atual->proximo. Ao final do uso, percorra novamente liberando cada nó.

7.2 Laboratório 3 — cadastro em lista

Modele um cadastro de alunos em lista encadeada. Implemente inserir, buscar por matrícula, remover e liberar. Especifique o comportamento para lista vazia, matrícula inexistente e falha de alocação.

8. Comparação com outros paradigmas

Paradigma	Unidade central	Como a computação é descrita	Exemplos
Imperativo	comando e estado	mudanças sucessivas de memória	C, Pascal, Fortran
Orientado a objetos	objeto e método	estado encapsulado e mensagens	C++, Java, Python
Funcional	função e expressão	avaliação e composição, idealmente sem efeitos	Haskell, Lisp, ML
Lógico	fato, regra e consulta	dedução a partir de relações	Prolog

As fronteiras não são absolutas. Linguagens modernas combinam características: C++ associa orientação a objetos a controle imperativo; Python oferece funções de primeira classe e mutabilidade; Lisp e ML podem incluir mecanismos imperativos. A comparação deve observar o modelo dominante e as garantias oferecidas, não apenas o nome da linguagem.

9. Projeto integrador

Desenvolva uma aplicação de linha de comando em C para gerenciar uma coleção de registros. O projeto deve conter menu, funções de responsabilidade única, persistência opcional em arquivo, busca, ordenação, validação, tratamento de erros e liberação completa dos recursos.

Critérios sugeridos

- Correção funcional e tratamento de casos de fronteira.
- Clareza da decomposição em funções e módulos.
- Uso apropriado de tipos, const, ponteiros e structs.
- Ausência de acessos inválidos, vazamentos e comportamentos indefinidos conhecidos.
- Testes demonstráveis e documentação das decisões.

10. Exercícios de revisão

1. Explique, com suas palavras, a diferença entre ambiente, memória e estado.
2. Determine o valor final de x: `int x = 2; x = x + 3; x *= 2;`
3. Escreva uma função que conte quantos valores pares existem em um vetor.
4. Por que uma função que precisa alterar o ponteiro inicial de uma lista recebe um `No **`?
5. Identifique dois riscos no uso de strings em C e proponha uma prevenção para cada um.
6. Implemente uma função recursiva para somar os valores de 1 até n.
7. Compare atribuição imperativa com avaliação de uma expressão funcional.
8. Descreva um algoritmo para remover o primeiro nó de uma lista, tratando lista vazia.
9. Proponha testes de fronteira para um programa de classificação de médias.
10. Relacione uma vantagem e uma desvantagem do controle explícito de memória em C.

Gabarito comentado

1. Ambiente associa nomes a entidades; memória guarda valores em localizações; estado é a configuração atual dessas associações e conteúdos.
2. x termina em 10: $2 + 3 = 5$ e $5 * 2 = 10$.
3. Percorra o vetor com um índice e incremente o contador quando `v[i] % 2 == 0`; valide n antes de acessar.
4. Porque a função precisa escrever no objeto ponteiro que representa a cabeça; um ponteiro simples alteraria apenas sua cópia local.
5. Acesso fora dos limites e falta de terminador; use limites explícitos, `fgets` e validação.
6. Caso-base $n \leq 0$ retorna 0; caso recursivo retorna $n + \text{soma}(n - 1)$.
7. A atribuição muda uma localização do estado; a avaliação funcional enfatiza o valor da expressão e evita, em sua forma pura, efeitos observáveis.
8. Se a cabeça for `NULL`, nada ocorre; caso contrário, guarde o segundo nó, libere o primeiro e atualize a cabeça.
9. Teste abaixo, exatamente em e acima dos limites de cada faixa, além de entrada não numérica.
10. Vantagem: eficiência e controle; desvantagem: maior risco de vazamentos, acessos inválidos e dependência da representação.

Glossário

Termo	Definição
Atribuição	Comando que calcula um valor e o armazena em uma localização.
Array	Coleção contígua de elementos do mesmo tipo, indexada.
Efeito colateral	Alteração observável no estado ou no ambiente além do valor produzido.
Escopo	Região do programa em que um nome pode ser referenciado.
Heap	Área usada para objetos de duração dinâmica.
Ponteiro	Valor que representa o endereço de um objeto ou função.
Recursão	Definição ou execução que se refere a uma instância do próprio problema.
Stack	Estrutura associada a chamadas e objetos de duração automática.
Struct	Tipo composto cujos campos podem ter tipos diferentes.
Tipo	Conjunto de valores e operações permitidas.

Referências

GABBRIELLI, M.; MARTINI, S. Programming Languages: Principles and Paradigms. 2. ed. Cham: Springer, 2023. Livro de referência da disciplina. Seções consultadas: controle de execução e atribuição; abstração de controle; estruturação de dados; aspectos históricos das linguagens.

PLANO DE ENSINO. Paradigmas de Programação. Bacharelado em Ciência da Computação, 2026/2. Faculdade Impacta Tecnologia. Documento fornecido para a elaboração desta apostila.

KERNIGHAN, B. W.; RITCHIE, D. M. The C Programming Language. 2. ed. Prentice Hall, 1988.

SEBESTA, R. W. Concepts of Programming Languages. 11. ed. Pearson, 2015.